

FILE HANDLING SAMPLE QUESTION

Suppose you have an IMDB movies dataset in CSV format. Each line contains the following movie information:

- `Series_Title` -> Title of the movie
- `Released_Year` -> Year at which that movie released
- `Genre` -> Genre of the movie
- `IMDB_Rating` -> Rating of the movie at IMDB website
- `Director` -> Name of the Director
- `Star1` -> Name of Actor1
- `Star2` -> Name of Actor2

You are expected to write a function, `movie_func(file_str, director_str, star_str, genre_str)` that returns a list of `Series_Titles` of the movies where `star_str` has acted (either as `Star1` or as `Star2`) or the movies were directed by the `director_str`.

The `file_str` argument is the file name given as a string, `director_str` is the name of the director, `star_str` argument is the name of the star you are searching for and `genre_str` is a movie genre. For testing your code, you may use the provided `"data.csv"` file.

The columns are separated by a `'|'` character. The content of the sample file you receive looks like the below:

```
Series_Title|Released_Year|Genre|IMDB_Rating|Director|Star1|Star2
12 Angry Men|1957|Crime,Drama|9.0|Sidney Lumet|Henry Fonda|Lee J. Cobb
The Lord of the Rings: The Return of the
King|2003|Action,Adventure,Drama|8.9|Peter Jackson|Elijah Wood|Viggo
Mortensen
Pulp Fiction|1994|Crime,Drama|8.9|Quentin Tarantino|John Travolta|Uma Thurman
Schindler's List|1993|Biography,Drama,History|8.9|Steven Spielberg|Liam
Neeson|Ralph Fiennes
Inception|2010|Action,Adventure,Sci-Fi|8.8|Christopher Nolan|Leonardo
DiCaprio|Joseph Gordon-Levitt
Fight Club|1999|Drama|8.8|David Fincher|Brad Pitt|Edward Norton
```

Function Definition

As said, your function must return a list of `Series_Titles` of the movies where `star_str` has acted (either as `Star1` or as `Star2`) or the movies were directed by the `director_str` and, in addition to this, the movie must include the given `genre_str`.

Note that the genre field may contain one or many genres which are separated by comma(s) ', '. Hence, you need to split the genre field and check for a match.

Hint:

- You can use built-in string methods. Especially the `.split()`, `.lower()` and `.strip()` functions might be useful.
- Be aware of newline symbols `'\n'` at the end of each line. You are expected to handle them. You can call the `"strip()"` function to remove `"\n"` at the end of each line.

Notes:

- Your function shall receive its data via its parameters only. In other words, your solution must NOT contain the `input()` function.
- Your function shall return its result, the IMDB ratings, as a list. It should not print anything.
- Any return value that doesn't conform with the expected output type will be graded as zero.
- Function name must be `movie_func`.
- There won't be any erroneous test cases. Therefore you do not need to check for empty or missing fields etc.

SAMPLE RUN:

```
>>> movie_func("data.csv", "Martin Scorsese", "Anne Hathaway", "Family")
```

```
[]
```

```
>>> movie_func("data.csv", "Sidney Lumet", "Edward Norton", "Drama")
```

```
['12 Angry Men', 'Fight Club', 'American History X', 'Network', 'Dog Day  
Afternoon', 'The Verdict', 'Serpico', 'The Illusionist', '25th Hour']
```

Solution

```
def movie_func(file_str, director_str, star_str, genre_str):  
    f = open(file_str, "r")  
    lines = f.readlines()  
    movies = []  
    for line in lines:  
        args = line.strip().split('|') # call strip to remove "\n" and then  
split each line  
  
        if director_str == args[4] or star_str == args[5] or star_str ==  
args[6]:  
            genres = args[2].split(',') # split genre field
```

```
        for genre in genres:
            if genre_str == genre:
                movies.append(args[0])
    return movies
```